

SYSTEM DESIGN

Days 11-20: Intermediate Concepts

Book 2 of the System Design Roadmap

Building on the fundamentals with real-life examples

Prerequisite: Days 1-10 (Beginner Guide) - scaling, load balancing, caching, databases, CAP theorem, message queues, CDNs.

What You Will Learn

Day 11	API Design - REST vs GraphQL vs gRPC
Day 12	Consistent Hashing - Scaling Without Chaos
Day 13	Rate Limiting & Throttling - Protecting Your System
Day 14	Proxies, Reverse Proxies & API Gateways
Day 15	Database Indexing & Query Optimization
Day 16	Distributed Transactions - 2PC & the Saga Pattern
Day 17	Leader Election & Consensus (Paxos/Raft, Simply)
Day 18	Event-Driven Architecture & Pub/Sub
Day 19	Observability - Logs, Metrics, Traces & Alerts
Day 20	Security Fundamentals + Capstone Design: WhatsApp

This second book assumes you're comfortable with the basics from Days 1-10: client-server, scaling, load balancing, caching, SQL/NoSQL, replication/sharding, CAP theorem, message queues, and CDNs. These next 10 days go one level deeper into how real production systems are actually built and kept safe under pressure.

API Design - REST vs GraphQL vs gRPC

How clients and servers agree on a common language

An API (Application Programming Interface) is the contract that defines how a client asks a server for something. Choosing the right style of API affects performance, flexibility, and how easy the system is to evolve over time.

REST

REST organizes everything around "resources" (like `/users/123` or `/orders/45`) and standard actions (GET, POST, PUT, DELETE). It's simple, widely understood, and works well with caching - but it can require multiple round trips if a screen needs data from several resources at once.

GraphQL

GraphQL lets the client describe exactly what fields it needs in a single request, even if that data comes from several underlying sources. This avoids over-fetching or under-fetching data, which is especially useful for mobile apps with limited bandwidth.

gRPC

gRPC is built for fast, structured communication between internal services (not typically used directly by a browser). It uses a compact binary format instead of text-based JSON, making it significantly faster - ideal for service-to-service calls inside a large system.

■ Real-Life Analogy

REST is like ordering from a fixed restaurant menu - you pick from pre-defined combos (endpoints), and sometimes you get more or less than you actually wanted. GraphQL is like a buffet where you fill your own plate with exactly the items you want, nothing more. gRPC is like a kitchen's internal walkie-talkie system between chefs - extremely fast and efficient, but not meant for customers to use directly.

■ Real-World Example

Facebook created GraphQL because its mobile app screens needed data from dozens of different backend resources, and making dozens of separate REST calls was too slow on mobile networks. Internally, companies like Google use gRPC extensively for the thousands of services that talk to each other behind the scenes.

■ Key Takeaway

Use REST for simple, cacheable, public-facing APIs. Use GraphQL when clients need flexible, precisely-shaped data from multiple sources. Use gRPC for fast internal service-to-service communication where every millisecond counts.

Today's practice: Open your browser's developer tools (Network tab) on any website and look at the API requests being made. Are they REST-style URLs?

Consistent Hashing - Scaling Without Chaos

How systems add or remove servers without reshuffling everything

On Day 6 you learned about sharding - splitting data across multiple servers. But a simple approach like `server = user_id % number_of_servers` causes a huge problem: if you add or remove even one server, almost every piece of data has to move to a new server, because the number you're dividing by changed. **Consistent hashing** solves this.

How It Works

Imagine a circle (a "hash ring") with numbers going from 0 to a very large maximum, then wrapping back to 0. Both servers and data items are placed on this ring based on a hash of their name or ID. Each data item is stored on the first server found when moving clockwise around the ring. When a server is added or removed, only the small portion of the ring near that server is affected - everything else stays exactly where it was.

■ Real-Life Analogy

Picture a large round dining table with numbered seats going all the way around in a circle. Guests (data) sit in the nearest empty seat clockwise from where their name falls. If one extra chair is added between two seats, only the one guest next to that gap needs to shift over - everyone else at the table stays exactly where they were sitting.

■ Real-World Example

Amazon's DynamoDB and many distributed caching systems like Memcached clusters use consistent hashing so that when engineers add more servers during a traffic spike (like a big sale), the system doesn't have to painfully re-shuffle almost all of its existing data.

■ Key Takeaway

Consistent hashing minimizes data movement when servers are added or removed, which is essential for scaling distributed caches and databases smoothly, without downtime or massive data transfers.

Today's practice: Imagine a round table with 100 numbered seats and 4 hosts seated at 4 points around it. If a 5th host is added, sketch out how few guests actually need to change seats.

Rate Limiting & Throttling - Protecting Your System

Making sure no single user (or attacker) can overwhelm everyone else

Rate limiting restricts how many requests a client (a user, an IP address, or an API key) can make within a given time window - for example, 100 requests per minute. It protects a system from being overwhelmed by a single misbehaving client, an accidental infinite loop in someone's code, or a deliberate attack.

Common Approaches

- **Fixed Window** - allow N requests per fixed time block (e.g., per minute), then reset.
- **Sliding Window** - a smoother version that avoids sudden bursts right at the edge of each window.
- **Token Bucket** - each client has a "bucket" of tokens that refills steadily; every request costs one token, and requests are rejected once the bucket is empty.
- **Leaky Bucket** - requests are processed at a fixed, steady rate no matter how fast they arrive, smoothing out bursts.

■ Real-Life Analogy

Think of a nightclub with a bouncer at the door. Only a certain number of people are let in per minute, no matter how large the crowd outside grows. This keeps the club from getting dangerously overcrowded, even if a huge group suddenly shows up all at once.

■ Real-World Example

Twitter/X limits how many tweets you can post per hour, and most public APIs (like weather or maps APIs) cap free users at a certain number of requests per day - both to keep the service fair for everyone and to prevent abuse.

■ Watch Out For

Rate limiting needs to be applied at the right layer - often at the load balancer or API gateway (covered tomorrow) - so that blocked requests are rejected early and cheaply, before they waste resources on your actual application servers.

■ Key Takeaway

Rate limiting isn't about being unfriendly to users - it's a safety valve that keeps a system fair, stable, and protected from being knocked over by one client's traffic spike.

Today's practice: Think of a real queue-based rule you've seen in daily life (e.g., "max 2 items per customer during a sale") and map it to one of the four rate limiting strategies above.

Proxies, Reverse Proxies & API Gateways

The middlemen that make large systems manageable

A **proxy** sits between a client and the internet, forwarding the client's requests on its behalf - often used for privacy or to bypass restrictions. A **reverse proxy** does the opposite: it sits in front of your servers and forwards incoming requests to the right backend, hiding the servers' details from the outside world. A load balancer (Day 3) is actually a specialized type of reverse proxy.

API Gateway

An API gateway is a more advanced reverse proxy placed in front of a microservices architecture (Day 9). Instead of clients calling dozens of different microservices directly, they call one single gateway, which handles routing, authentication, rate limiting, and combining responses from multiple services into one.

■ Real-Life Analogy

A proxy is like a personal assistant who makes calls on your behalf so the other party never sees your number. A reverse proxy is like a hotel receptionist: guests never call individual room extensions directly - they call the front desk, and the receptionist routes the call to the right room. An API gateway is that same receptionist, but also checking guests' ID at the door, limiting how many calls each guest can make, and sometimes combining messages from multiple departments into one summary for the guest.

■ Real-World Example

Netflix's API gateway sits in front of hundreds of internal microservices. When your TV app requests the home screen, the gateway quietly calls several services (recommendations, artwork, subtitles) and combines everything into one response, so your TV only has to make a single request.

■ Key Takeaway

Reverse proxies and API gateways hide the internal complexity of a system from the outside world, and give you one central place to handle routing, security, and rate limiting instead of repeating that logic in every single service.

Today's practice: Think of a hotel front desk, a company reception, or a call center's IVR system - identify what role is played by the "reverse proxy" in each scenario.

Database Indexing & Query Optimization

Why some queries are instant and others take forever

As tables grow to millions of rows, searching through every single row to find what you need (a "full table scan") becomes painfully slow. An **index** is a special data structure that lets the database jump almost directly to the matching rows, instead of checking every one.

■ Real-Life Analogy

Think of a thick textbook with no index at the back. To find every mention of "photosynthesis," you'd have to flip through every single page. Now imagine the same book with a proper index at the back listing "photosynthesis: pages 45, 112, 230." You jump straight to the right pages instead of reading the whole book - that's exactly what a database index does for a query.

The Trade-off

Indexes aren't free - they take up extra storage, and every time you insert, update, or delete a row, the database also has to update the index, which slightly slows down writes. This is why databases aren't indexed on every single column - engineers pick indexes based on which columns are searched or filtered on most often.

- Index columns that are frequently used in WHERE clauses, JOINs, or sorting (e.g., user_id, email).
- Avoid indexing columns that change constantly or are rarely searched - the maintenance cost isn't worth it.
- A "composite index" can cover multiple columns together for queries that filter on more than one field at once.

■ Real-World Example

An e-commerce site searching for "all orders placed by user_id 4521" across tens of millions of orders would be unusably slow without an index on user_id - with one, the database finds those rows almost instantly instead of scanning the entire orders table.

■ Key Takeaway

Indexes trade a little extra storage and slightly slower writes for dramatically faster reads. Good system design means indexing thoughtfully - based on your app's real, frequent query patterns, not indexing everything blindly.

Today's practice: Think of any app you use with a search bar. What field (username, product name, order ID) is probably indexed to make that search fast?

Distributed Transactions - 2PC & the Saga Pattern

Keeping multiple services in agreement when one action touches several of them

In a microservices world (Day 9), a single user action - like "place an order" - might need to update several independent services: payment, inventory, and shipping. What happens if the payment succeeds but the inventory update fails? Distributed transaction patterns solve this coordination problem.

Two-Phase Commit (2PC)

A coordinator first asks every service involved, "are you ready to commit this change?" (the *prepare* phase). Only if every single service says yes does the coordinator tell them all to actually commit (the *commit* phase). If even one says no, everyone is told to roll back instead. This guarantees consistency, but it's slow, and if the coordinator crashes mid-process, services can get stuck waiting.

The Saga Pattern

Instead of one big all-or-nothing transaction, a saga breaks the process into a sequence of smaller, independent steps, each with its own "undo" action (a compensating transaction). If a later step fails, the system runs the compensating actions for the steps that already succeeded, rolling things back gradually rather than instantly.

■ Real-Life Analogy

2PC is like a wedding planner calling the caterer, the venue, and the band, asking each "can you confirm for this date?" and only finalizing the booking once every single one says yes - if even one says no, the whole booking is cancelled. The Saga pattern is more like booking each vendor one at a time: if the band cancels after the venue and caterer are already booked, you don't panic - you simply call the venue and caterer back to cancel those specific bookings (the compensating actions).

■ Real-World Example

When you book a flight and hotel together on a travel site, if the hotel booking fails after the flight was already booked, a saga-style system automatically triggers a "cancel flight" compensating action instead of leaving you with a half-completed, inconsistent booking.

■ Key Takeaway

2PC guarantees strict consistency but is slow and fragile at scale. The Saga pattern is more resilient and widely used in modern microservices, trading strict all-or-nothing guarantees for gradual, recoverable steps.

Today's practice: Think through booking a multi-leg trip (flight + hotel + car rental). If one leg fails after the others succeeded, what would the "compensating action" be for each of the other two?

Leader Election & Consensus (Paxos/Raft, Simply)

How a group of servers agrees on "who's in charge" without a boss

In a distributed system with many servers, it's often useful to have one server act as the "leader" for a period of time - for example, the one server allowed to accept writes in a replicated database (Day 6). But how do a group of independent servers, with no central boss, agree on who that leader should be - especially if some servers might crash or lose network connection at any time? This is what **consensus algorithms** like Raft and Paxos solve.

The Basic Idea (Raft, Simplified)

- Every server starts as a "follower." If a follower doesn't hear from a leader for a while, it assumes there is no leader and becomes a "candidate."
- The candidate asks every other server to vote for it. If it gets votes from a majority of servers, it becomes the new leader.
- The leader regularly sends "heartbeat" signals to the others to prove it's still alive. If those stop, a new election begins.

■ Real-Life Analogy

Imagine a group of friends on a road trip with no fixed leader, but they need one person to make quick decisions (which route to take, where to stop). If the current decision-maker goes silent for too long (phone dies), someone else speaks up and asks, "should I take over?" If most of the group agrees, that person becomes the new decision-maker. As long as more than half the group agrees on who is leading at any moment, the trip keeps moving smoothly even if people's phones occasionally die.

■ Real-World Example

Systems like etcd (used heavily inside Kubernetes) and CockroachDB use the Raft consensus algorithm to reliably elect a leader among a cluster of servers, ensuring the cluster keeps working correctly even if some servers crash or lose connectivity.

■ Key Takeaway

Consensus algorithms let a group of unreliable, independent servers agree on a single source of truth - like who the current leader is - even when some of them fail. This is the invisible foundation underneath many modern distributed databases and coordination tools.

Today's practice: Think of a group project or family decision where no one person is officially "in charge," but the group still needs someone to make quick calls. How does the group informally handle it if that person goes quiet?

Event-Driven Architecture & Pub/Sub

Letting services react to what happened, instead of asking for it directly

On Day 8 you learned about message queues, where one producer sends a task to one consumer. **Publish-Subscribe (Pub/Sub)** takes this further: a service *publishes* an event (like "order placed") without knowing or caring who's listening, and any number of other services can *subscribe* to that event and react to it independently.

Why It's Powerful

This decouples services from each other. The "order placed" event might be consumed by the inventory service, the email service, the analytics service, and a fraud-detection service - all at once, all independently - without the order service needing to know any of them exist. New services can even be added later, simply by subscribing to events that are already being published.

■ Real-Life Analogy

Think of a school's announcement speaker system instead of the principal personally walking to every classroom to deliver a message. The principal (publisher) makes one announcement over the speakers (the event), and any classroom (subscriber) that cares about it - the sports team, the music club, the canteen - reacts on its own. The principal doesn't need to know which classrooms are listening or how many there are.

■ Real-World Example

When you place an order on Amazon, that single "order placed" event is picked up independently by the warehouse system (to start packing), the notifications system (to email you), the recommendations system (to update what it suggests next), and the finance system (to log the sale) - all without those systems being directly wired together.

■ Key Takeaway

Event-driven, pub/sub architecture makes large systems easier to extend and evolve, because services react to what happened rather than being tightly coupled to each other through direct calls.

Today's practice: Think of one action in an app you use (e.g., "post uploaded", "payment received") and list at least three different parts of the system that probably need to react to that single event.

Observability - Logs, Metrics, Traces & Alerts

How engineers actually know what's happening inside a live system

Once a system is running in production with real users, you need ways to see inside it without stopping it - this is called **observability**. It rests on three main pillars.

- **Logs** - detailed, timestamped text records of individual events ("user 4521 logged in at 10:03am"). Great for investigating a specific incident after the fact.
- **Metrics** - numeric measurements tracked over time (requests per second, average response time, error rate). Great for spotting trends and triggering alerts.
- **Traces** - follow a single request as it travels through multiple services, showing exactly where time was spent. Essential in microservices, where one user action might touch a dozen services.

■ Real-Life Analogy

Running a system without observability is like flying a plane with the cockpit windows painted over and no instrument panel. Logs are like the flight's black box recorder - detailed records you can review after something goes wrong. Metrics are like the speedometer and altitude gauge - quick numbers you glance at constantly. Traces are like air traffic control following one specific flight's entire journey, city by city, to see exactly where it got delayed.

■ Real-World Example

When a food delivery app has a sudden spike in "payment failed" errors, engineers first notice it through a metrics dashboard (error rate spiking), get paged by an automated alert, then use traces to find which specific service in the payment flow is failing, and finally dig into that service's logs to find the exact cause.

■ Key Takeaway

You can't fix - or even notice - what you can't see. Observability tools (like Prometheus, Grafana, and distributed tracing systems) are what let engineers keep large, complex systems healthy in production, and react quickly when something breaks.

Today's practice: Think of a car's dashboard. Which part is like a "metric" (speedometer), which is like a "log" (trip computer history), and which resembles a "trace" (GPS route history)?

Security Fundamentals + Capstone: Designing WhatsApp

Keeping systems safe, then combining everything you've learned

Core Security Concepts

- **Authentication** - proving who you are (e.g., logging in with a password or fingerprint).
- **Authorization** - determining what you're allowed to do once you're identified (e.g., only admins can delete a post).
- **Encryption in transit** - scrambling data as it travels over the network (HTTPS/TLS) so it can't be read if intercepted.
- **Encryption at rest** - scrambling data while it's stored on disk, so a stolen hard drive doesn't expose raw data.

■ Real-Life Analogy

Authentication is like a security guard checking your ID at the building entrance. Authorization is the guard then checking whether your ID badge allows you into the server room or only the lobby. Encryption in transit is like sending a locked box through the mail instead of a postcard anyone can read. Encryption at rest is keeping that same box locked even while it's sitting inside a warehouse.

Capstone: Designing WhatsApp (High Level)

Requirements: Users send real-time text messages, see delivery/read receipts, and messages must be delivered even if the recipient is briefly offline.

High-Level Design: Clients connect to chat servers using persistent connections (WebSockets) for instant, two-way communication (Day 1 concepts, evolved). A **load balancer** (Day 3) distributes connections across many chat servers. Messages for offline users are placed in a **message queue** (Day 8) and delivered once the recipient reconnects. Chat history is stored in a **sharded database** (Day 6) - sharded by conversation ID so each server only needs a portion of the total data. A **pub/sub system** (Day 18) notifies all of a user's active devices simultaneously when a new message arrives.

Security: Messages are end-to-end encrypted, meaning even WhatsApp's own servers can't read message content - only the sender and recipient's devices hold the keys needed to decrypt it.

■ Key Takeaway

Real production systems always combine multiple concepts together - scaling, queues, sharding, pub/sub, and security all working in concert. The individual pieces you've learned across these 20 days are the vocabulary; combining them thoughtfully for a specific problem is the actual skill.

Where to Go From Here

- Practice designing more systems end-to-end: an Instagram feed, a ride-sharing app, a video streaming service, a ticket booking system.
- Read engineering blogs from real companies to see how these exact patterns are applied at scale.
- Time yourself explaining a design out loud in under 20 minutes - this mirrors real interview conditions.

You've now covered both the fundamentals and the intermediate building blocks used in real, large-scale systems. The next step is repetition: design something new every few days, and notice how much faster the pieces click together each time.